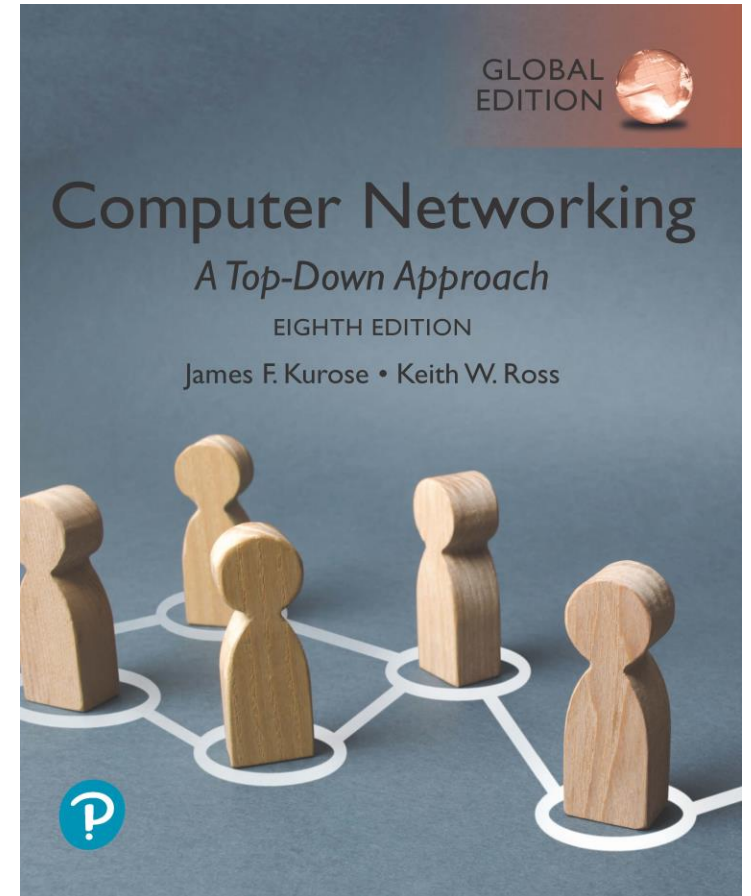


Chapter 3

Transport Layer



Computer Networking: A Top-Down Approach

8th Edition, Global Edition

Jim Kurose, Keith Ross

Copyright © 2022 Pearson Education Ltd

Chapter 3: roadmap

- Transport-layer services
- Multiplexing and demultiplexing
- **Connectionless transport: UDP**
- Principles of reliable data transfer
- Connection-oriented transport: TCP
- Principles of congestion control
- TCP congestion control
- Evolution of transport-layer functionality



UDP: User Datagram Protocol

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out-of-order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- no connection establishment (which will add RTT delay)
- simple: no connection state at sender, receiver
- small header size
- no congestion control
 - UDP can blast away as fast as desired!
 - can function in the face of congestion

UDP: User Datagram Protocol

- UDP use:
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - *(En sannhet med modifikasjoner, tjenester som Netflix etc bruker TCP)*
 - DNS
 - SNMP
 - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
 - add needed reliability at application layer
 - add congestion control at application layer

UDP: User Datagram Protocol [RFC 768]

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

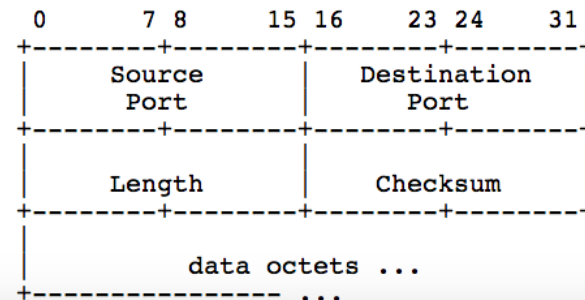
User Datagram Protocol

Introduction

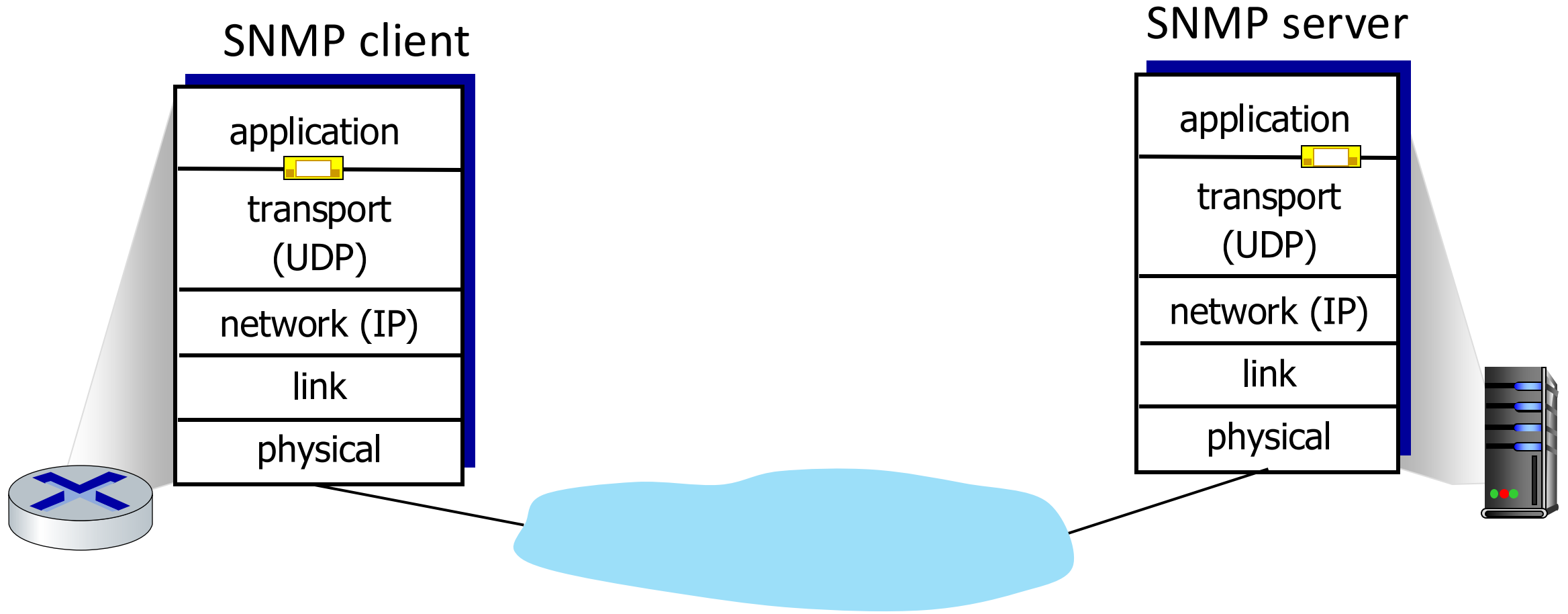
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

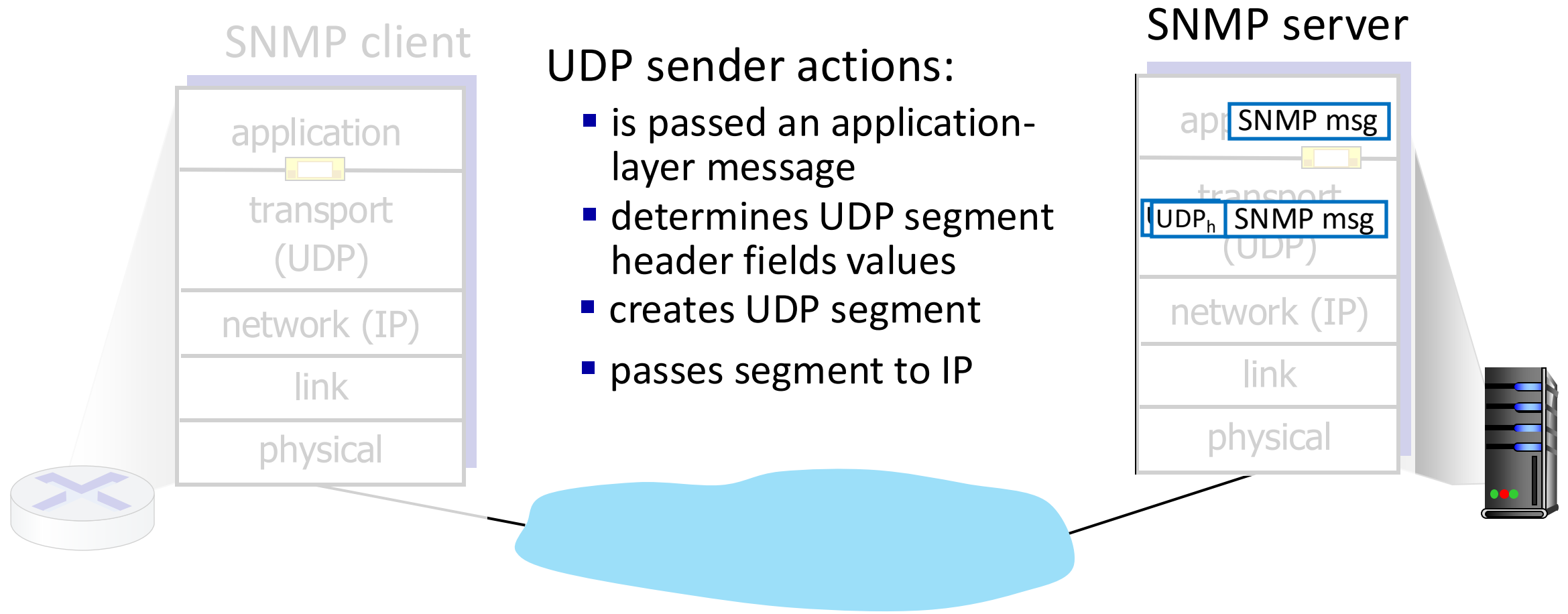
Format



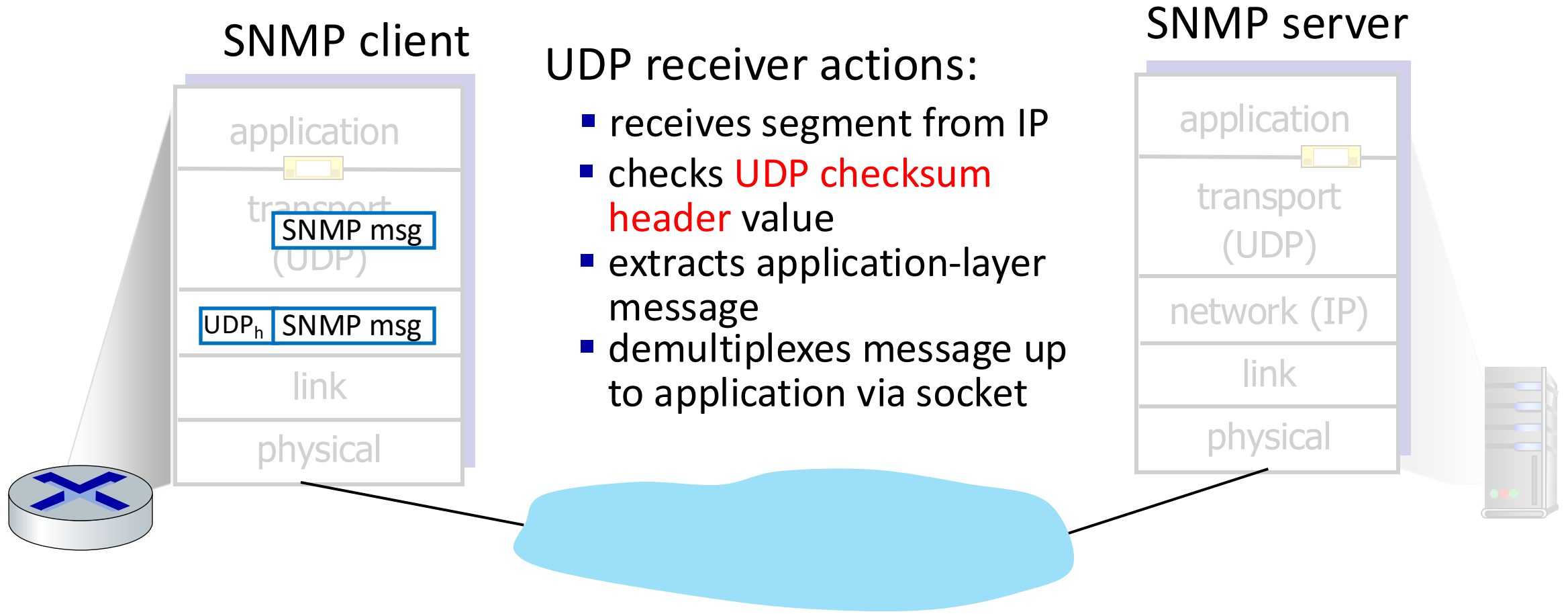
UDP: Transport Layer Actions



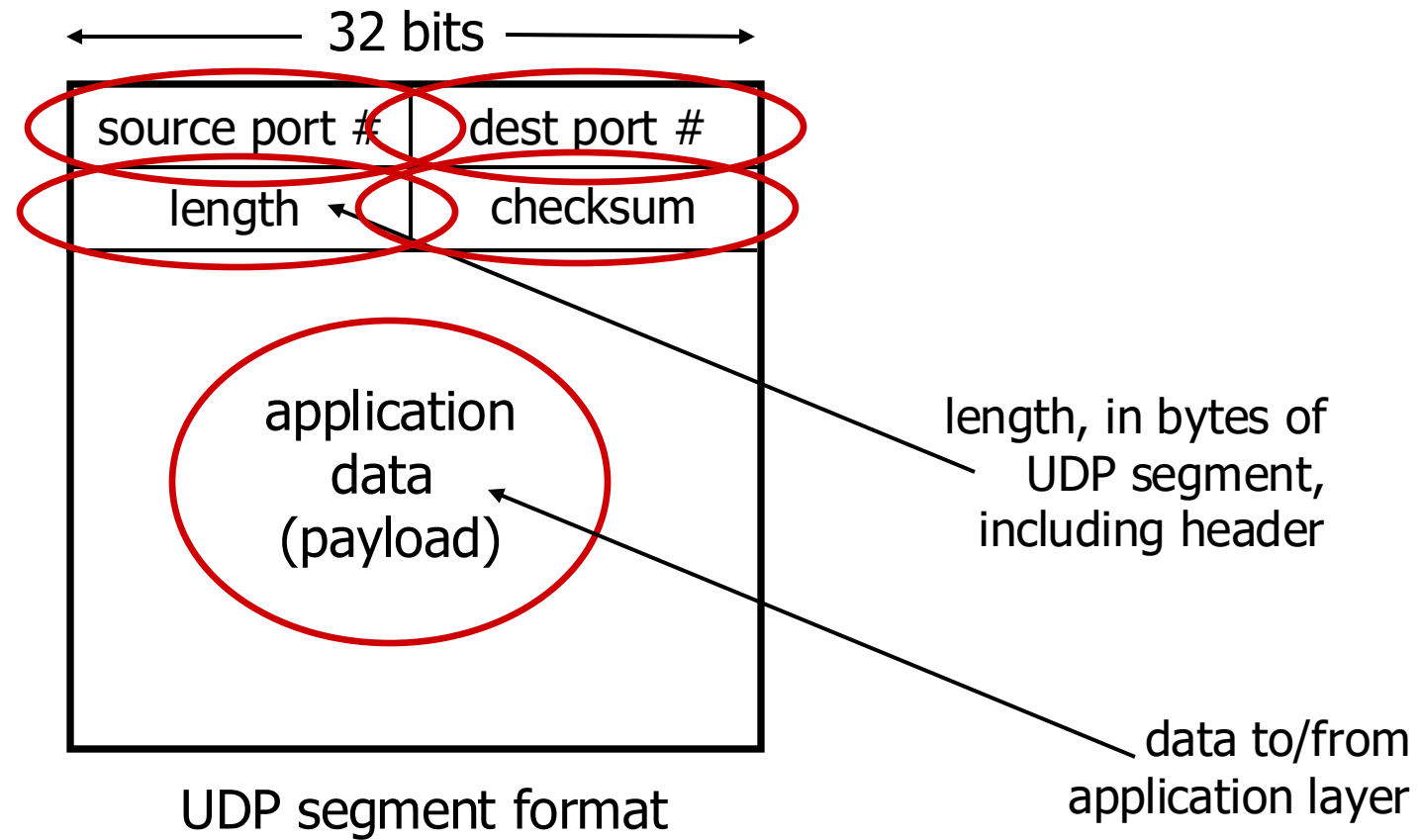
UDP: Transport Layer Actions



UDP: Transport Layer Actions

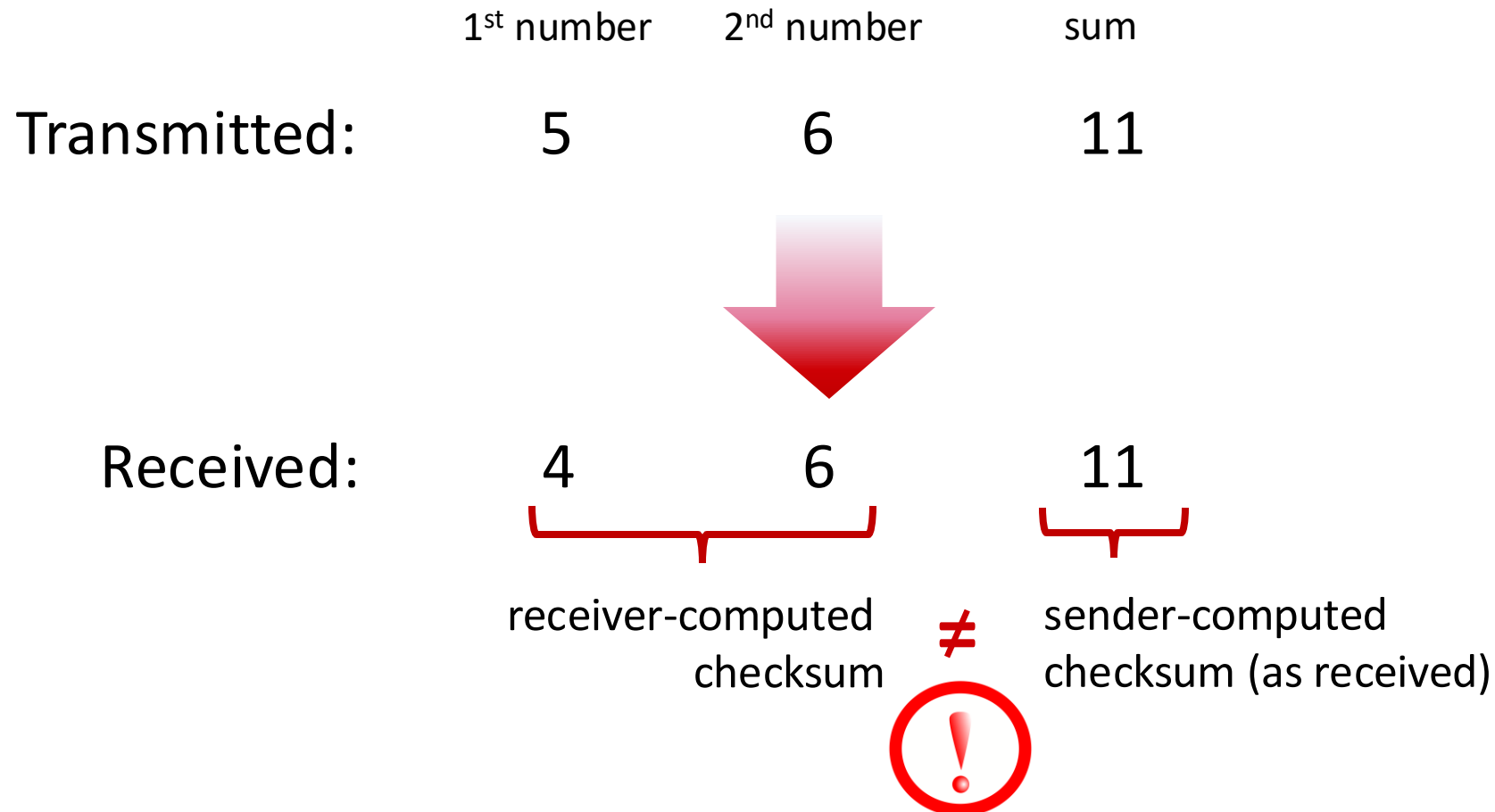


UDP segment header



UDP checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment



Internet checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

sender:

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- **checksum:** addition (**one's complement sum**) of segment content
- checksum value put into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - not equal - error detected
 - equal - no error detected. *But maybe errors nonetheless?* More later

$$0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1 \text{ og } 1 + 1 = 10$$

Internet checksum: an example

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0	
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

$$0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1 \text{ og } 1 + 1 = 10$$

Internet checksum: weak protection!

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Even though numbers have changed (bit flips), *no* change in checksum!

Hvorfor "one's complement" på sum for å lage checksum ?

The complement is taken to make checksum validation simpler — instead of calculating the checksum again and then comparing the calculated value with the checksum field in the header (which is in the middle of the summed data), it is possible to sum (using one's complement arithmetic) all 16-bit words in the header (including the checksum word) and compare the result with zero.

Suppose the data we want to send consists of the following three words:

```
0110011001100000
0101010101010101
1000111100001100
```

Husk denne enkle binære matten :-)

$0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1$ og $1 + 1 = 10$

Adding the first two words:

```
0110011001100000
0101010101010101
```

```
1011101110110101
```

Then, adding the third word to this sum:

```
1011101110110101
1000111100001100
```

```
0100101011000001
```

However, there is a carry out, which we need to add to the final sum again:

```
0100101011000001 + 1 =
0100101011000010
```

Finally, we take the 1's complement of the final sum, which in this case, becomes:

```
1011010100111101
```

Internet checksum with 3 words of 16 bit

At the receiver side, all the 16-bit data chunks are added again, with any overflow being wrapped around. The checksum is also added to the final result. The answer at the receiver's side should consist of all ones. If a single bit is zero, it means an error occurred during transmission.

```
0110011001100000
0101010101010101
1000111100001100
```

```
1011010100111101
```

```
1111111111111111
```

Summary: UDP

- “no frills” protocol:
 - segments may be lost, delivered out of order
 - best effort service: “send and hope for the best”
- UDP has its plusses:
 - no setup/handshaking needed (no RTT incurred)
 - can function when network service is compromised
 - helps with reliability (checksum)
- build additional functionality on top of UDP in application layer (e.g., HTTP/3)